

AI-3016 マイクロソフト認定: Azure での AI の概要

コース実施日 : May, 2026

貴重なお時間を割いて講義にご参加くださり有難ございます。AI は日々進化しております。本資料は現時点での情報としてご使用いただき、ぜひ学びを続け AI を使用してのソリューション開発をお楽しみください！

※本資料は 2026 年 5 月 11 日時点の情報となりますことご注意ください。

はじめに

✓ 講義で使用する PowerPoint は著作権の関係で配布できませんが、学習内容は以下のリンクから復習をしたり知識を深掘して頂けます。

<https://learn.microsoft.com/ja-jp/training/paths/develop-generative-ai-apps/>

✓ 今後配信予定のマイクロソフト認定資格については[こちらのリンク](#)からご参照頂けます。

※英語版ですが、Browser 翻訳で日本語に変更可能です

✓ 仮想環境を使用して実技演習が可能となっております。有効期限内であれば1つのモジュールにつき最大 10 回まで演習が可能です。アクセスキーのコードは後ほど講師より案内がございます。

進化する AI

昨今のAIの進化に関して

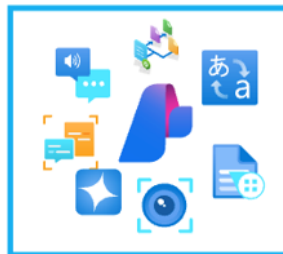
各種機能に特化したAI
(単一タスク向けアプリケーション)



(例)
画像認識1、物体検出、音声解析、翻訳、
感情分析、OCR など

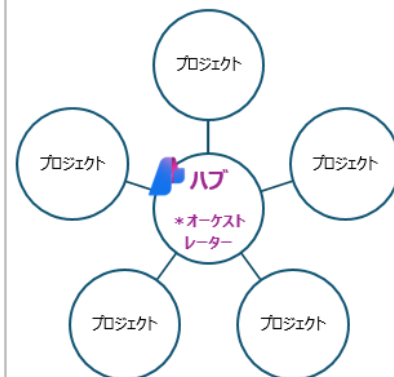
— 機能特化型 AI —

生成AIがアプリケーションを呼び出し、一部の業務を**自動化**



— AI エージェント —
* ツール実行型 *

生成AIが複数のアプリケーションを連携させ、AIが**自律的に行動**
(計画→実行→振り返り)



— エージェントAI —
* 目的達成型 *

責任ある AI の原則について

責任ある AI の原則：公平性、信頼性と安全性、プライバシーとセキュリティ、包括性、透明性、説明責任。

AI を活用してローンの申請の結果を判断する例で解説：

原則	概要	ビジネス例
公平性	特定の属性による不当な差別や偏りを避け、公平な判断を行う	年齢や性別などに依存せず、収入・返済履歴など正当な指標でローン審査する
信頼性と安全性	想定外の入力や誤動作に備え、安定して安全に動作する	ローン審査 AI に人によるレビューや異常検知の仕組みを組み込む
プライバシーとセキュリティ	個人情報適切に管理・保護し、漏えいや不正利用を防ぐ	申請者データを暗号化し、業務上必要な人のみがアクセス可能にする
包括性	多様な利用者が不利なく利用できるよう配慮する	多言語対応やアクセシビリティに配慮したローン申請サポートを提供する
透明性	AI が用いられていることや判断の考え方を利用者に示す	ローン審査で AI を使用していることと、主な判断要素を申請者に説明する
説明責任	AI の判断結果や運用について、人が責任を持つ	AI による誤った審査結果が発生した場合、組織として説明し是正対応を行う

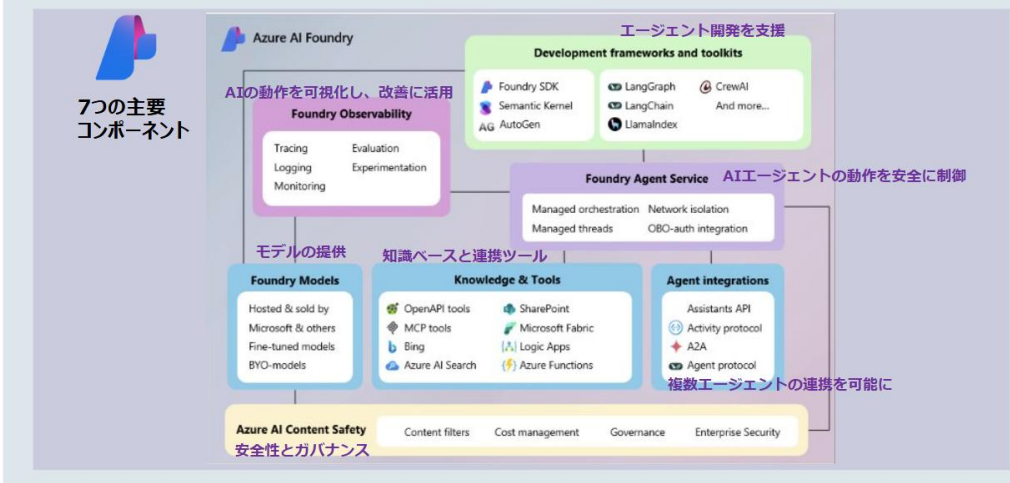
✓ [責任ある AI の原則とアプローチ | Microsoft AI](#)

✓ [責任ある AI - Training | Microsoft Learn](#)

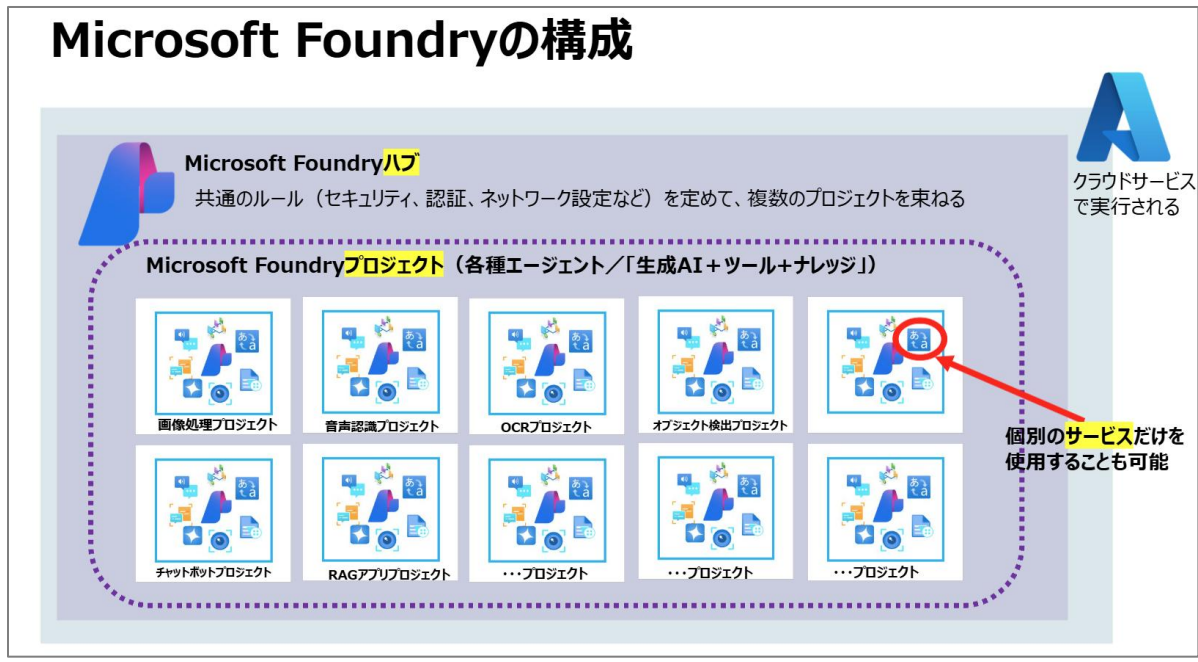
Microsoft Foundry について

Microsoft Foundryとは

生成AIを使って会話ベースでAIエージェント同士や外部ツールと連携でき、人の判断を含む複雑な業務も自律的に処理できる時代の到来。
モデル選択・必要なアプリケーションの構築・AIに参照させる情報の構築・AIに取らせるアクションフローの設計を企業レベルのガバナンスとセキュリティが標準搭載されている安全・安心な環境で一元管理できるのがMicrosoft Foundryのすごいところ。



Microsoft Foundryの構成



Microsoft Foundry プロジェクト

- AI ワークロードの作業単位（Workspace）で、Vision、Document Intelligence、生成 AI、Agents 等を同じプロジェクト内で組み合わせて使えるのが前提

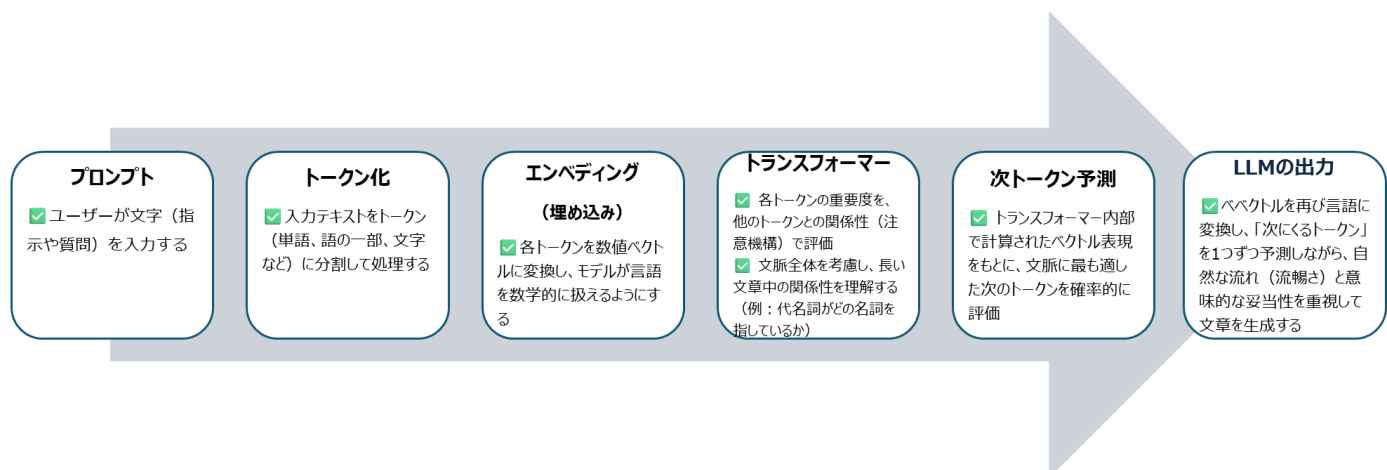
💡 予備知識 💡

Microsoft Foundry ハブ ベースのプロジェクト

- 複数プロジェクトで共通の基盤（モデル、接続、ストレージ、セキュリティ、ガバナンス）を共有するための上位コンテナ
（例）生成 AI のエージェントが会話を通じて Vision や他 AI を呼び出すような**複合シナリオ**では、同じハブ配下で同一の接続・ストレージ・ポリシーを共有した方が、設計・運用・セキュリティが一気に楽になります。結果として、再利用・一貫性・ガバナンスが効く

生成 AI の構造理解

*これは、生成 AI の一種である大規模言語モデル（LLM）によるテキスト生成プロセスを簡略化して示したものです。



トークン化の例 : [Tokenizer - OpenAI API](#)

OpenAI Platform

Learn more.

You can use the tool below to understand how a piece of text might be tokenized by a language model, and the total count of tokens in that piece of text.

GPT-5.x & O1/3 GPT-4 & GPT-3.5 (legacy) GPT-3 (legacy)

今日は晴れて良い天気です。

Clear Show example

Tokens 9 Characters 13

今日は晴れて良い天気です。

Clear Show example

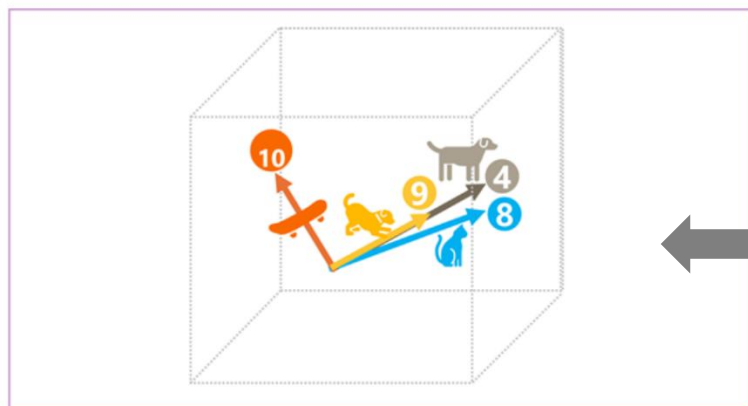
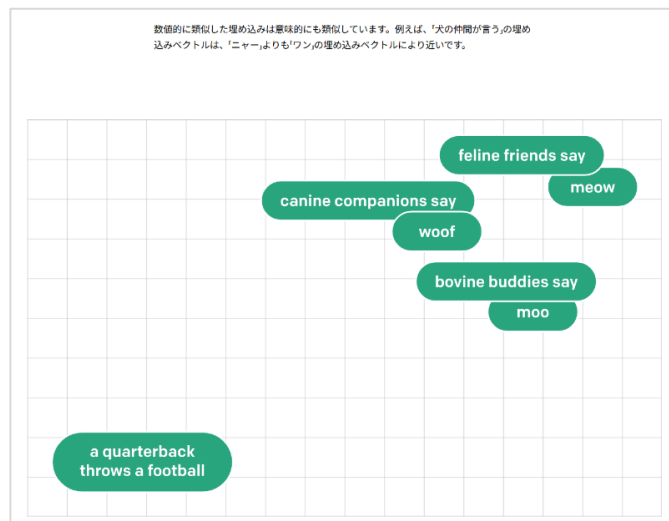
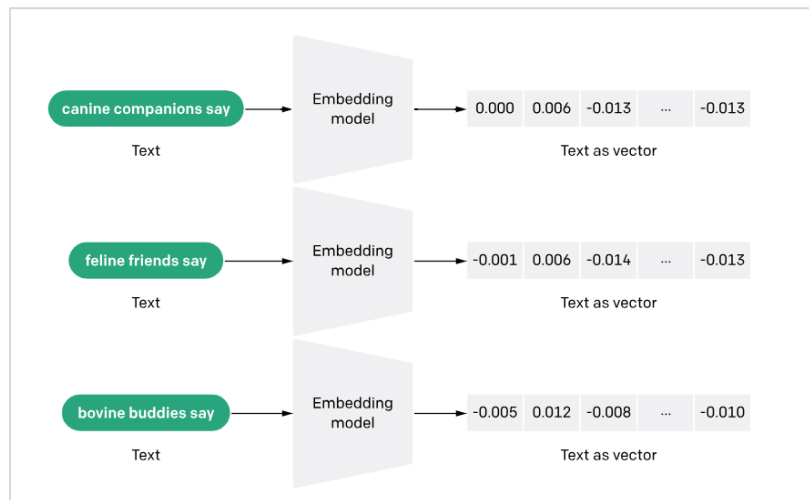
Tokens 9 Characters 13

[170411, 123139, 185887, 35015, 3826, 867, 25717, 15121, 788]

Text Token IDs

エンベディング（埋め込み）の例 :

[Introducing text and code embeddings | OpenAI](#)



ベクトル空間のイメージ

<https://learn.microsoft.com/ja-jp/training/modules/fundamentals-generative-ai/3-language-models?pivots=text>

生成 AI モデルを選ぶ際のポイント

デル選択要因	要因の解説（なぜ重要か）	ビジネスでの具体例
ベンダーとライセンス	どのベンダーのモデルか、データの扱い・利用条件・商用利用の可否がビジネス要件に合うかが重要。法務・セキュリティ・契約条件に直結する。	金融・公共分野では「Microsoft管理下でデータが保持されるモデル」を選択。外部学習に使われないことが必須条件。
サイズとコスト	モデルサイズが大きいほど性能は上がる傾向があるが、推論コスト・レイテンシ・スケールコストが増える。用途に見合ったサイズ選択が重要。	社内FAQや要約用途では小〜中規模モデルを採用し、月間利用コストを抑制。高度な分析は限定的に大型モデルを使用。
パフォーマンスメトリック	精度だけでなく、応答速度、安定性、再現性など、実運用でのKPIを満たすかが重要。	コールセンター支援では「正確さ」より「即時応答」が重要なため、低レイテンシなモデルを優先。
デプロイオプション	クラウド、リージョン、ネットワーク分離、VNet対応など、どこでどう動かせるかが運用・セキュリティに影響。	社内システム連携のため、VNet内デプロイ可能なモデルを選択し、外部インターネット接続を遮断。
リージョン別の提供状況	モデルがどのリージョンで利用可能かは、データ主権・遅延・法規制に影響する。	日本の個人情報情報を扱うため、日本リージョンで提供されているモデルのみを選択。海外リージョンは不可。

エンドポイントにモデルをデプロイする種類

デプロイの種類	技術的な意味（要約）	ビジネスでの最適なユースケース例
グローバル標準	世界共通で使えるリアルタイム推論	グローバル向け FAQ チャットボット、Web サイトの問い合わせ対応、社内 AI アシスタント
グローバル プロビジョニング済み	事前に容量確保された高スループット	世界同時展開のカスタマーサポート、繁忙期が予測できる EC サポート AI
グローバル バッチ	非同期・大量処理	全社文書の要約・分類、過去ログの分析、ナレッジ一括生成
データゾーン標準	EU/US などデータ所在を限定	欧州向け顧客データ分析、金融・公共分野の AI 利用
データゾーン プロビジョニング済み	データ所在 + 性能保証	規制対応が必要な業界での高負荷 AI（保険審査、与信チェック）
データゾーン バッチ	データゾーン内での非同期処理	EU 顧客データの一括 OCR 処理、契約書解析
標準（リージョン）	特定リージョン内での利用	国内向け業務 AI、PoC、部門単位の AI 活用
リージョン プロビジョニング済み	リージョン内で性能確保	国内コールセンターAI、業務システム連携 AI
Developer	開発・評価向け	モデル評価、PoC、プロンプト検証、開発者の実験用途

エンドポイントと SDK の選択

観点	Foundry プロジェクト Endpoint + Foundry SDK	Unified OpenAI Endpoint + OpenAI SDK
----	--	---

位置づけ	Foundry ネイティブ。AI アプリ基盤として使う	OpenAI 互換。LLM API としてシンプルに使う
Endpoint	Foundry Project Endpoint	Unified OpenAI Endpoint
主な SDK	Microsoft Foundry SDK (azure-ai-projects)	OpenAI SDK
認証	Microsoft Entra ID (企業向け標準)	Entra ID または API キー
使用 API	Responses API (推奨)	Responses API または Chat Completions
得意分野	エージェント、評価、監視、ガバナンス	チャット、要約、生成を素早く実装
Playground との関係	Playground で試した設計をそのまま本番へ	Playground で挙動確認 → 軽量実装
向いている開発段階	本番・部門横断・長期運用	PoC・小規模・既存アプリ統合

ChatCompletions と Responses API の比較

※以下のコードは説明を分かりやすくするために、Copilot を活用して作成したサンプルです。実際に使用する際は、Foundry Portal など最新情報をご確認ください。

[Chat Completions]

```
1 messages = [{"role": "system", "content": SYSTEM}]
2
3 # Turn 1
4 messages.append({"role": "user", "content": "東京から日帰りで自然が多いおすすめは?"})
5 r1 = client.chat.completions.create(model=MODEL, messages=messages)
6 a1 = r1.choices[0].message.content
7 print("A1:", a1)
8 messages.append({"role": "assistant", "content": a1})
9
10 # Turn 2
11 messages.append({"role": "user", "content": "予算1万円以内でモデルコースにして。"})
12 r2 = client.chat.completions.create(model=MODEL, messages=messages)
13 a2 = r2.choices[0].message.content
14 print("A2:", a2)
15 messages.append({"role": "assistant", "content": a2})
16
17 # Turn 3
18 messages.append({"role": "user", "content": "混雑が少ない時間帯のコツも1つ添えて。"})
19 r3 = client.chat.completions.create(model=MODEL, messages=messages)
20 a3 = r3.choices[0].message.content
21 print("A3:", a3)
22 messages.append({"role": "assistant", "content": a3})
23
```

[Responses API]

```
1 last_id = None
2
3 # Turn 1
4 r1 = client.responses.create(
5     model=MODEL,
6     instructions=SYSTEM, # system相当 (簡潔に見せたいならここが最適) [3]
7     input="東京から日帰りで自然が多いおすすめは?",
8     previous_response_id=last_id
9 )
10 print("A1:", r1.output_text)
11 last_id = r1.id
12
13 # Turn 2
14 r2 = client.responses.create(
15     model=MODEL,
16     instructions=SYSTEM,
17     input="予算1万円以内でモデルコースにして。",
18     previous_response_id=last_id
19 )
20 print("A2:", r2.output_text)
21 last_id = r2.id
22
23 # Turn 3
24 r3 = client.responses.create(
25     model=MODEL,
26     instructions=SYSTEM,
27     input="混雑が少ない時間帯のコツも1つ添えて。",
28     previous_response_id=last_id
29 )
30 print("A3:", r3.output_text)
31 last_id = r3.id
```

共通「ツール」

※Tool は、事前学習済みモデルの知識だけでは届かない現実世界の情報や操作につなぐ拡張機能です。

ツール	何ができるか（要点）	ビジネスシナリオ例
code_interpreter	Python コードを生成・実行し、計算・分析・可視化を行う	営業・企画分析 ：売上 CSV を基に、月次成長率・前年差分・グラフを自動作成。 経営報告 ：KPI データから要約とチャートを生成。 Excel 代替 ：大量データを手作業なしで迅速に分析。
web_search	インターネット上の最新情報や外部情報を検索する	市場調査 ：競合ニュース、価格改定、業界トレンドを迅速に把握。 営業準備 ：商談前に顧客企業の直近リリースや決算情報を確認。 FAQ 対応 ：最新制度や法改正を踏まえて回答。
file_search	指定したナレッジ（社内文書・PDF など）を根拠付きで検索する	社内問い合わせ対応 ：就業規則・手順書・契約書を根拠に正確に回答。 カスタマーサポート ：製品マニュアルや FAQ を基に回答。 RAG シナリオ ：社内ナレッジを活用する AI を実現。
Function	アプリ側のカスタム関数や API を呼び出す	業務自動化 ：CRM への顧客登録、チケット作成、在庫照会を実行。 ワークフロー連携 ：承認申請、通知、DB 更新を自動化。 現実世界へのアクション ：AI が回答だけでなく処理も実行。

補足解説：Code Interpreter について

Code Interpreter で「自動化されること」

- エージェントが **Python コードを自動で作成**する
- 作成したコードを **サンドボックス（隔離）環境で実行**する
- 実行結果（数値・表・グラフ）を **その場で確認**する
- 結果を見て、エージェント自身が **コードを修正して再実行**する
- この流れを **人が指示しなくても繰り返す（反復的な試行錯誤）**

 **仮説 → 実行 → 修正 → 再実行** が自動化される

Code Interpreter が特に得意なタスク

- CSV / Excel などの **データ集計・分析**
- 平均・合計・割合などの **計算処理**
- グラフ・チャートなどの **可視化**
- 外れ値除外や条件変更をしながらの **試行錯誤型分析**

 人が Excel で手作業していた「考えながら回す作業」を肩代わり

自動化されないこと（重要）

- 何を分析したいかの目的設定
- 結果がビジネス的に正しいかの判断
- 外部システムや社内 DB への 直接アクセス
- 本番システムへの 即時反映・確定処理

💡 判断と意思決定は人、作業と試行錯誤は AI

RAG・Grounding に関して

Groundingについて

ユーザープロンプト

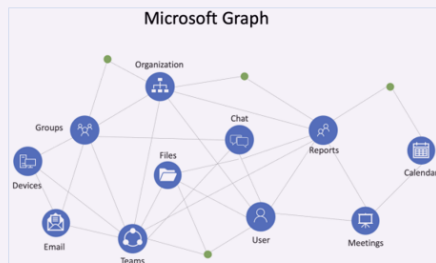


RAGによって追加された情報
位置情報
WEB検索結果



LLMへ渡す

グラウンディング用の “tool” となり得るもの



組織内の情報



WEB情報



モデルに取得して欲しい
ユニーク（オリジナル）
情報



地図情報（位置情報）

M365 Copilotの場合 = Retrieve Organizational Data + Search Up-to-date market data （retrieving and reasoning with information）

<https://colab.research.google.com/github/GoogleCloudPlatform/generative-ai/blob/main/gemini/grounding/intro-grounding-gemini.ipynb?hl=ja#scrollTo=5ebdda19afad>

RAG構成の考え方(例) - オーバーラップ

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

1 Introduction

Recurrent neural networks, long short-term memory [13] and gated recurrent [7] neural networks in particular, have been firmly established as state of the art approaches in sequence modeling and transduction problems such as language modeling and machine translation [35, 2, 5]. Numerous efforts have since continued to push the boundaries of recurrent language models and encoder-decoder architectures [38, 24, 15].

Recurrent models typically factor computation along the symbol positions of the input and output sequences. Aligning the positions to steps in computation time, they generate a sequence of hidden states h_t as a function of the previous hidden state h_{t-1} and the input for position t . This inherently sequential nature precludes parallelization within training examples, which becomes critical at longer sequence lengths, as memory constraints limit batching across examples. Recent work has achieved significant improvements in computational efficiency through factorization tricks [21] and conditional computation [32], while also improving model performance in case of the latter. The fundamental constraint of sequential computation, however, remains.

Attention mechanisms have become an integral part of compelling sequence modeling and transduction models in various tasks, allowing modeling of dependencies without regard to their distance in the input or output sequences [2, 19]. In all but a few cases [27], however, such attention mechanisms are used in conjunction with a recurrent network.

前のチャンクから **n%** をオーバーラップさせる

RAG構成の考え方(例) - 固定サイズ

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

1 Introduction

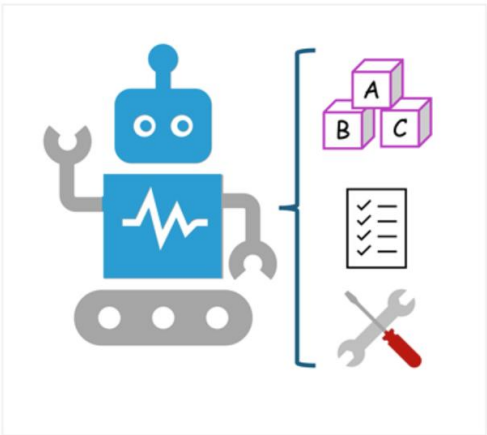
Recurrent neural networks, long short-term memory [13] and gated recurrent [7] neural networks in particular, have been firmly established as state of the art approaches in sequence modeling and transduction problems such as language modeling and machine translation [35, 2, 5]. Numerous efforts have since continued to push the boundaries of recurrent language models and encoder-decoder architectures [38, 24, 15].

Recurrent models typically factor computation along the symbol positions of the input and output sequences. Aligning the positions to steps in computation time, they generate a sequence of hidden states h_t as a function of the previous hidden state h_{t-1} and the input for position t . This inherently sequential nature precludes parallelization within training examples, which becomes critical at longer sequence lengths, as memory constraints limit batching across examples. Recent work has achieved significant improvements in computational efficiency through factorization tricks [21] and conditional computation [32], while also improving model performance in case of the latter. The fundamental constraint of sequential computation, however, remains.

Attention mechanisms have become an integral part of compelling sequence modeling and transduction models in various tasks, allowing modeling of dependencies without regard to their distance in the input or output sequences [2, 19]. In all but a few cases [27], however, such attention mechanisms are used in conjunction with a recurrent network.

固定サイズごとに分割

AI エージェントについて



AI エージェント - Training | Microsoft Learn

<例>

朝イチ「インボックス・トリージ」エージェント

未読メールを「要返信」「情報共有」「対応不要」に分類し、重要メールは要点 + 返信案まで作る *by Agent Builder

“業務横断” チケット処理エージェント（分類→調査→起票→返信）

問い合わせ内容を分類し、社内ナレッジ（file_search） + 必要なら計算/解析（code_interpreter） + 業務システム連携（function）で、解決案作成や起票まで一気通貫。 *by Microsoft Foundry

同期 vs 非同期

	同期ジョブ（Sync）	非同期ジョブ（Async）
レスポンス方法	1 回の呼び出しで結果が返る （リクエスト内で完結）	まず「ジョブ作成」→「完了待ち（状態確認）」→「結果取得」という段階型
適する処理の特徴	1 枚画像の特徴抽出など、短時間で終わる処理 （例：Caption/Tags/Objects/People/Read OCR などを 1 回で）	計算量が大きい・時間がかかる・大量ページ/大容量ファイルなど（例：文書の高解像 OCR、動画生成）
理由	ユーザー体験を止めないため 即時に返せる範囲の処理に最適化されている	時間と計算資源を要するため